

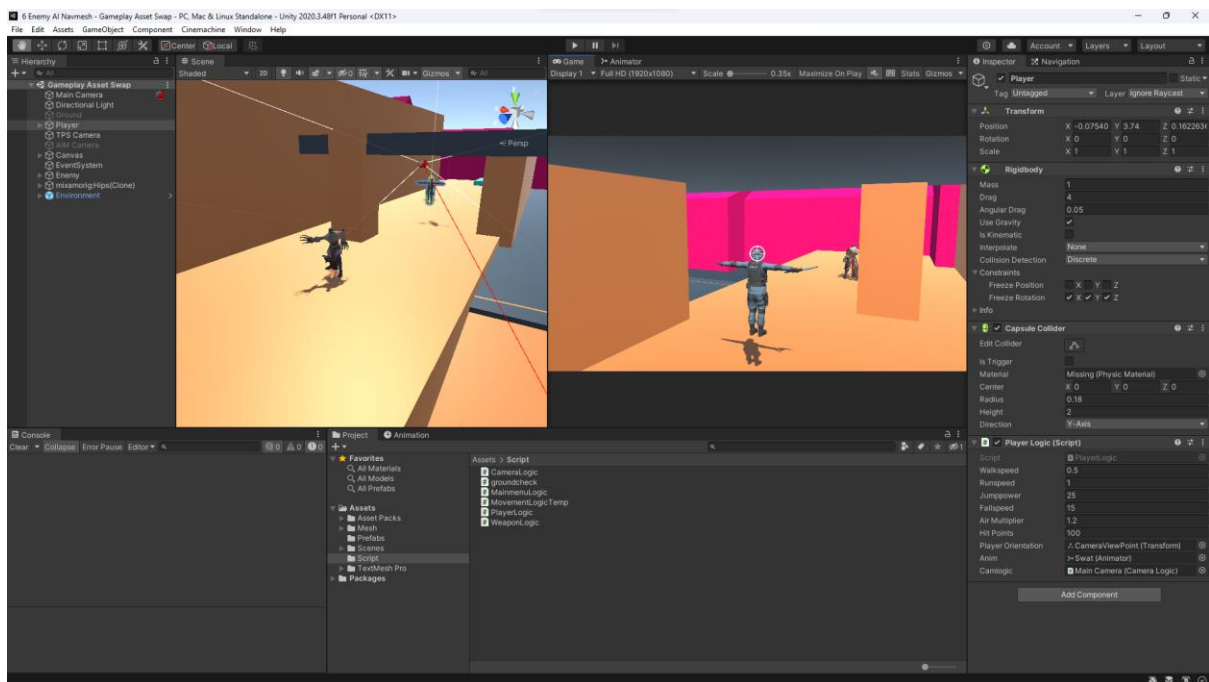
MODUL 6

PRAKTIKUM PEMROGRAMAN MULTIMEDIA & GAME

Enemy Logic & Navmesh

Pada Modul ini, Mahasiswa akan mempelajari tentang pembuatan logika **Enemy**. Logika ini berfungsi agar enemy dapat mendekati Player, dan akan menyerang jika sudah dekat dengan player, Langkah langkahnya adalah sebagai berikut.

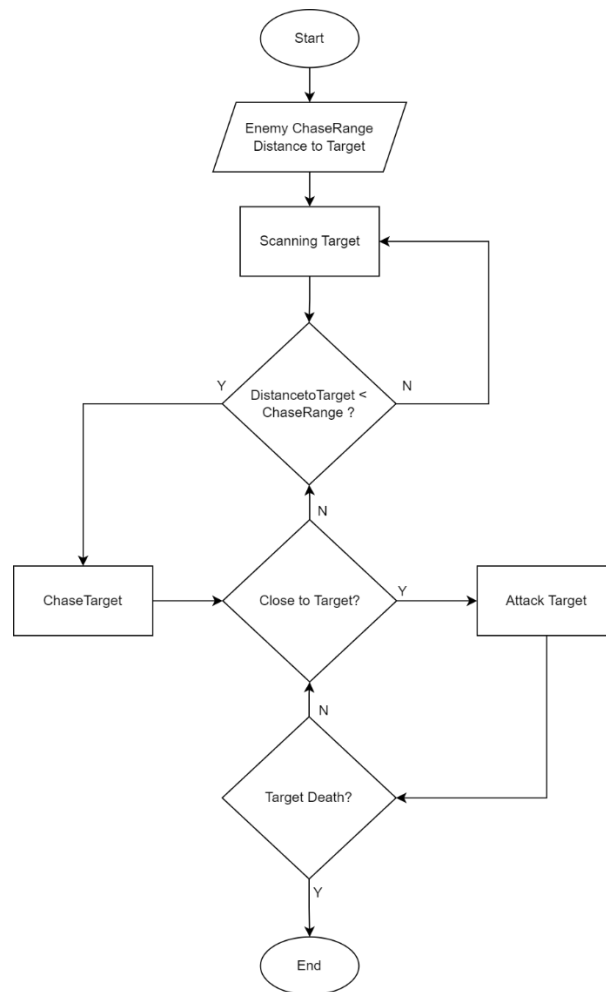
1. Buka Project Unity Modul 5 Anda (Player Attack Logic)



Pada Praktikum Modul 6, akan melanjutkan modul sebelumnya, jika sudah menyelesaikan Modul 5, anda bisa melanjutkan praktikum Modul 6, namun jika anda belum menyelesaikan Modul 5, maka selesaikanlah terlebih dahulu!

2. Path Finding Logic.

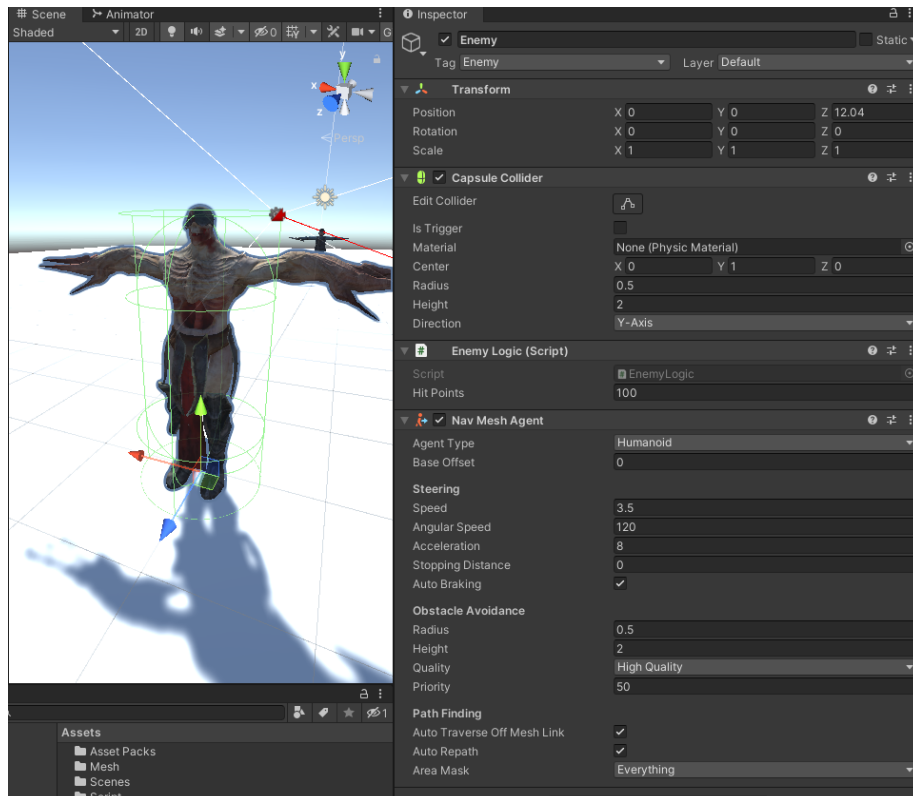
Logika yang akan digunakan pada Modul ini adalah, **Enemy** akan mendeteksi apakah ada Player yang mendekati dia, jika ada maka enemy akan mengejar Player, namun jika tidak terdeteksi, enemy akan tetap berada di state idle.



Algoritma yang digunakan untuk proses **Pathfinding** pada modul ini adalah **Navmesh**, Navmesh adalah sebuah fitur bawaan dari Unity untuk memberikan fungsi pathfinding kepada GameObject.

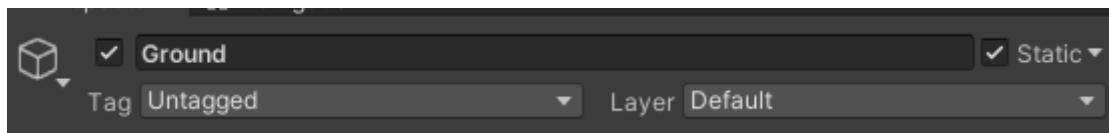
3. Unity Navmesh Component

Untuk menggunakan Navmesh, Langkah pertama yang harus dilakukan adalah. Menambahkan Component NavMesh Agent pada Enemy.

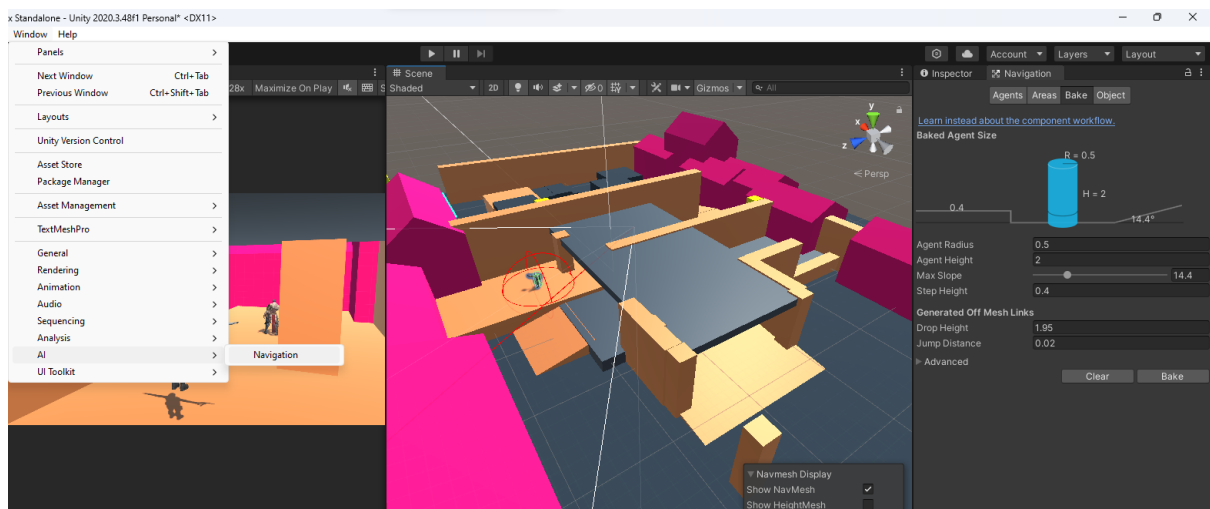


Selanjutnya adalah proses **Bake/Render** bidang yang akan menjadi tempat AI berjalan, langkahnya adalah sebagai berikut :

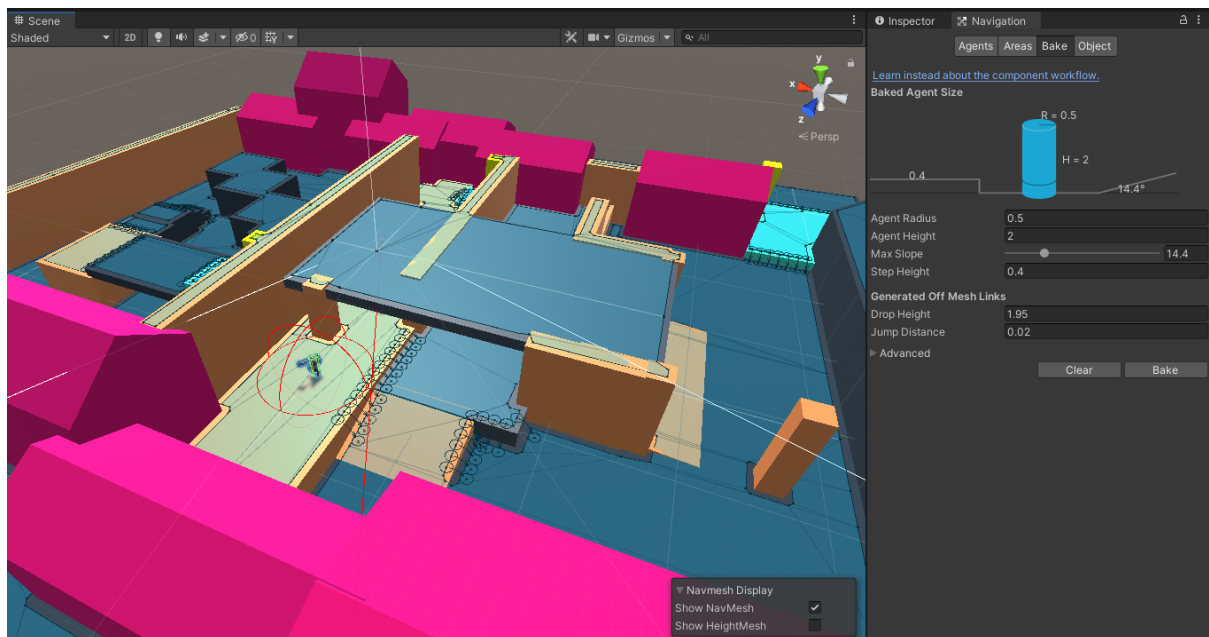
- Setting Bidang Yang dapat dilalui menjadi object **Static**, karena AI hanya akan merender bidang yang **Static**.



- Buka **Tab Navigation**, jika masih belum ada, buka tab tersebut seperti pada gambar dibawah.



- Masuk kedalam **opsi** Bake dan tekan tombol **Bake**. Jika ground memiliki highlight berwarna biru, maka konfigurasi sudah benar, jika belum lakukan pengecekan ulang!



4. Penulisan Script Enemy Logic

Setelah Penambahan Component NavMeshAgent pada enemy dan Bake bidang AI selesai dilakukan, Langkah selanjutnya adalah penulisan Script. Ikuti Langkah Langkah berikut :

- Penulisan Library dan Variable Global

```
using System.Collections;
using System.Collections.Generic;
using UnityEngine;
using UnityEngine.AI;

Unity Script (1 asset reference) | 2 references
public class EnemyLogic : MonoBehaviour
{
    public float hitPoints = 100f;
    public Transform target;
    public float ChaseRange;
    private NavMeshAgent agent;
    private float DistancetoTarget;
```

- Logika Dasar Mengejar Target

```
Unity Message | 0 references
private void Start()
{
    agent = this.GetComponent<NavMeshAgent>();
}

Unity Message | 0 references
private void Update()
{
    DistancetoTarget = Vector3.Distance(target.position, transform.position);

    if (DistancetoTarget <= ChaseRange)
    {
        ChaseTarget();
    }
}

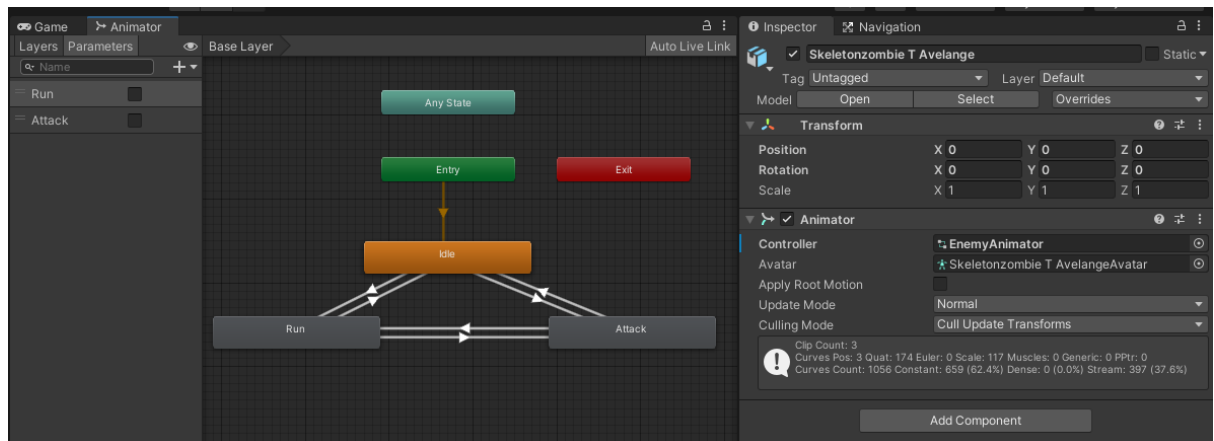
1 reference
public void ChaseTarget()
{
    agent.SetDestination(target.position);
}

Unity Message | 0 references
void OnDrawGizmosSelected()
{
    Gizmos.color = Color.red;
    Gizmos.DrawWireSphere(transform.position, ChaseRange);
}
```

5. Logika Berhenti dan Menyerang dan Setup Animator.

Langkah selanjutnya adalah mengkonfigurasi dan penulisan script untuk logika enemy menyerang karakter jika berada dijarak menyerang, dan akan mengerjar Kembali jika player sudah keluar dari jarak menyerang. Ikuti Langkah Langkah berikut.

- **Buat Animator Controller untuk Enemy.**
- **Setup Animator Controller seperti gambar dibawah.**



Beberapa animasi yang digunakan adalah **“Zombie idle”**, **“Zombie Attack”**, dan **“Zombie Running”**. Boleh menggunakan animasi berbeda.

Selanjutnya Masukkan logika kondisi seperti dibawah.

1. Idle => Run : Run = True.
2. Run => Idle : Run = False.
3. Idle => Attack : Attack = True.
4. Attack => Idle : Attack = False.
5. Run => Attack : Attack = True.
6. Attack => Run : Run = True.

Uncheck semua **has exit time** kecuali Attack => Idle dan Attack => Run.

- **Tulis Beberapa Deklarasi Variable Globala & Inisialisasi animator.**

```
public float hitPoints = 100f;
public Transform target;
public float ChaseRange;
private NavMeshAgent agent;
private float DistancetoTarget;
private Animator anim;

Unity Message | 0 references
private void Start()
{
    agent = this.GetComponent<NavMeshAgent>();
    anim = this.GetComponentInChildren<Animator>();
}
```

- Tambahkan Baris kode untuk Berlari dan menyerang

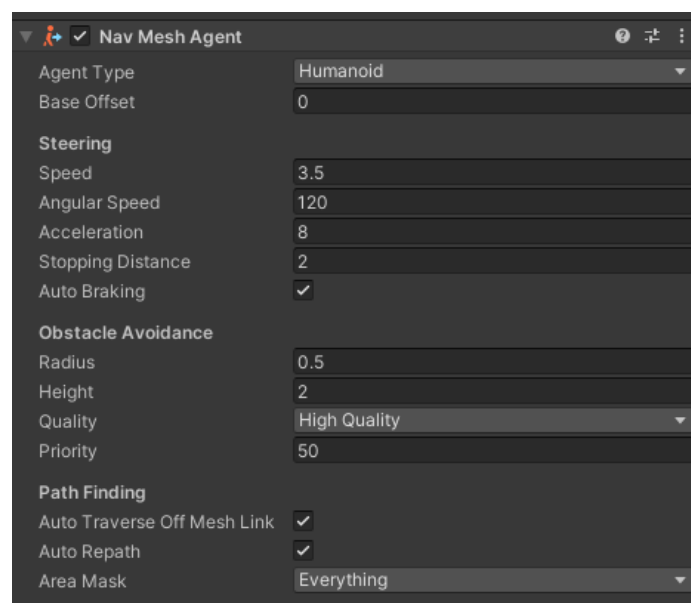
```
Unity Message | 0 references
private void Update()
{
    DistancetoTarget = Vector3.Distance(target.position, transform.position);

    if (DistancetoTarget <= ChaseRange)
    {
        if (DistancetoTarget > agent.stoppingDistance)
        {
            ChaseTarget();
        }
        else if (DistancetoTarget <= agent.stoppingDistance)
        {
            Attack();
        }
    }
}

1 reference
public void Attack()
{
    Debug.Log("attack");
    anim.SetBool("Run", false);
    anim.SetBool("Attack", true);
}

1 reference
public void ChaseTarget()
{
    agent.SetDestination(target.position);
    anim.SetBool("Run", true);
    anim.SetBool("Attack", false);
}
```

- Tulis Stopping Distance pada NavMeshAgent Enemy menjadi 2.



Jika sudah, Save Kodingan dan Kembali ke unity, lakukan pengujian, apakah **enemy mendekati** dan memainkan **animasi run** Ketika player masuk ke **ChaseRange**, dan apakah Enemy dapat **menyerang** Ketika sudah pada **stopping distance**? Jika sudah, maka konfigurasi dan kodingan sudah benar, jika masih belum, lakukan pengecekan ulang!

6. Face Target.

Pada step sebelumnya, enemy sudah dapat mengejar dan menyerang player, namun Ketika enemy sedang menyerang, enemy tidak dapat berotasi sesuai dengan arah player. Oleh karena itu akan ditambahkan beberapa baris script agar enemy dapat selalu menoleh ke arah player Ketika menyerang. Berikut adalah beberapa baris kode yang digunakan.

- Tambahkan variable global baru.

```
public class EnemyLogic : MonoBehaviour
{
    public float hitPoints = 100f;
    public float turnSpeed = 15f;
    public Transform target;
    public float ChaseRange;
    private NavMeshAgent agent;
    private float DistancetoTarget;
    private Animator anim;
```

- Tambahkan beberapa baris kode berikut.

```
Unity Message | 0 references
private void Update()
{
    DistancetoTarget = Vector3.Distance(target.position, transform.position);

    if (DistancetoTarget <= ChaseRange)
    {
        FaceTarget(target.position);
        if (DistancetoTarget > agent.stoppingDistance)
        {
            ChaseTarget();
        }
        else if (DistancetoTarget <= agent.stoppingDistance)
        {
            Attack();
        }
    }
}

1 reference
private void FaceTarget(Vector3 destination)
{
    Vector3 direction = (destination - transform.position).normalized;
    Quaternion lookRotation = Quaternion.LookRotation(new Vector3(direction.x, 0, direction.z));
    transform.rotation = Quaternion.Slerp(transform.rotation, lookRotation, Time.deltaTime * turnSpeed);
}
```

Jika sudah, Save Kodingan dan Kembali ke unity, lakukan pengetesan, apakah Enemy dapat menoleh kearah karakter Ketika menyerang atau tidak, jika bisa, maka kodingan sudah benar, jika belum lakukan pengecekan ulang!

7. Selesai Untuk Praktikum Modul 6

Praktikum Modul 6 dikatakan selesai jika praktikan menyelesaikan PathFinding menggunakan NavMesh, Memasang animasi Enemy Idle,Run, dan Attack. Dan Enemy dapat menghadap karakter Ketika sedang menyerang.

Tugas Modul 6

Animation Event & Combat Logic

1. Enemy Hit Connect Logic (Animation Event).

Fungsi EnemyHitConnect ini adalah untuk memasukkan logika apakah serangan enemy sudah mengenai player atau belum. Jika sudah maka player akan merespon, dan jika serangan tidak mengenai, player tidak akan merespon. Ikuti beberapa Langkah berikut :

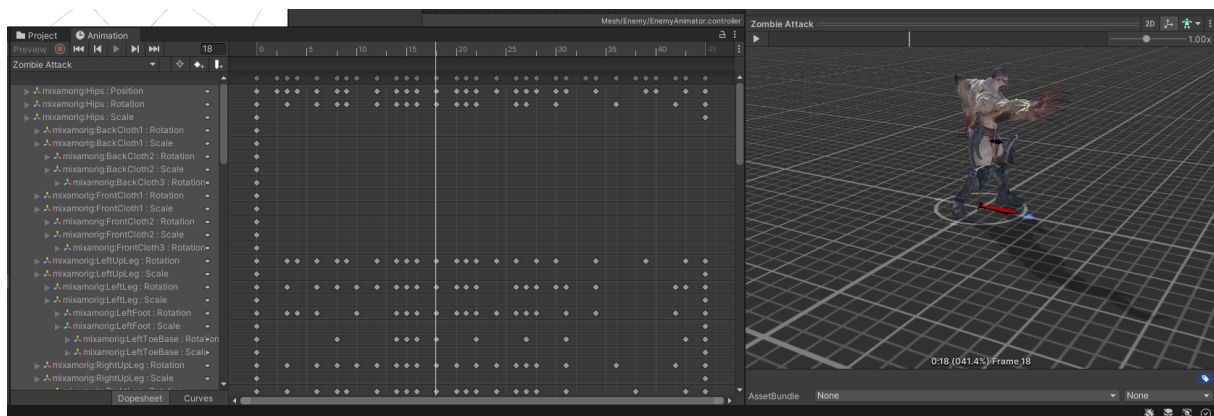
- **Rubah Nama Script MovementLogic Menjadi PlayerLogic.**

Ini dilakukan karena MovementLogic Sudah tidak hanya menampung script movement namun juga Logika Player. Jangan lupa merubah nama script di Visual Studio dan Unity. (perhatikan kapital, spasi, dan symbolnya). Dan fix kan beberapa error yang ada.

- **Tuliskan Baris Kode Hit Connect pada EnemyLogic.**

```
public void HitConnect()
{
    if (DistancetoTarget <= agent.stoppingDistance)
    {
        target.GetComponent<PlayerLogic>().PlayerGetHit(50f);
    }
}
```

- **Logika Zombie Attack.**

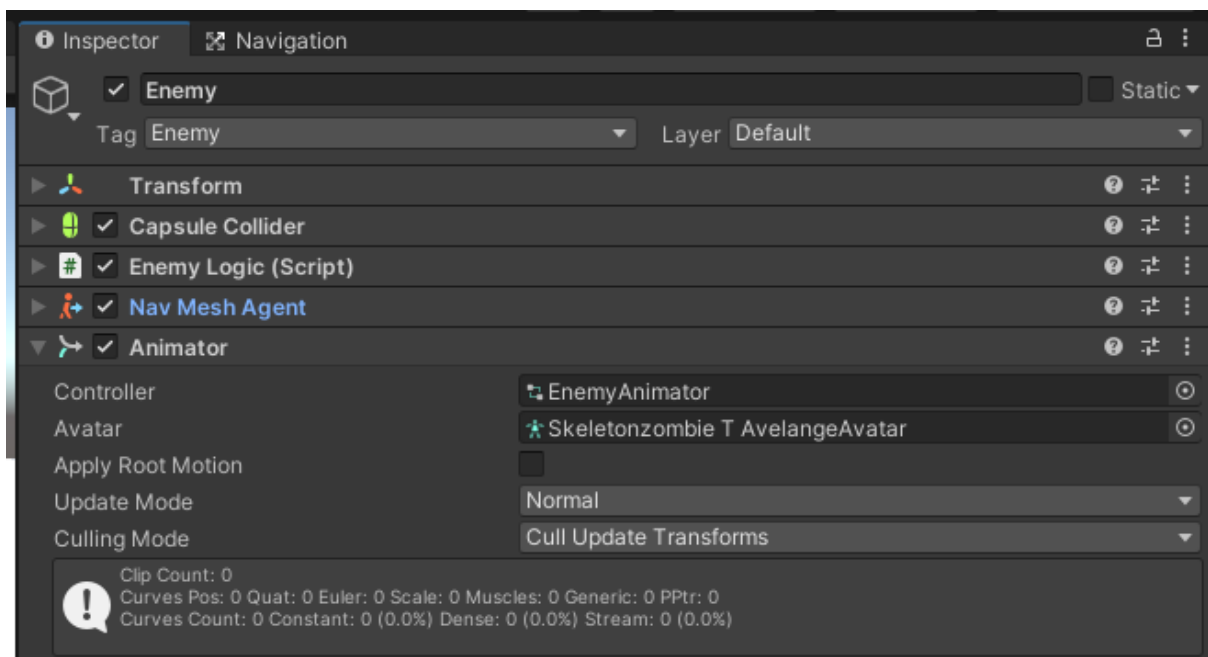
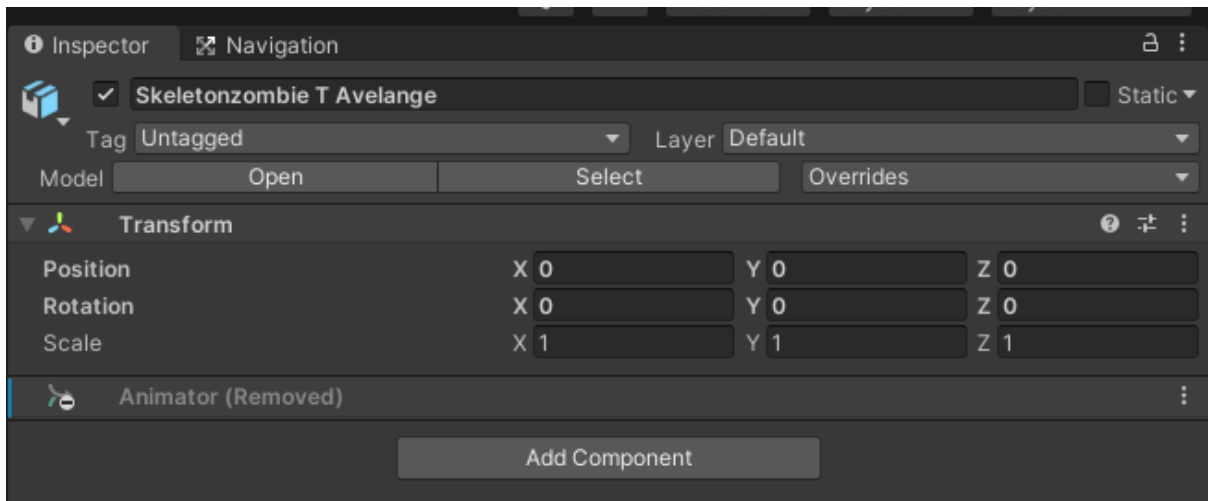


Logika menyerang, pada umumnya akan memberikan kerusakan Ketika Serangan menyentuh player, sedangkan pada animasi **Zombie Attack** Gerakan memukul baru terjadi Ketika **Frame 18**, oleh karena itu akan ditambahkan **Event HitConnect** Ketika ada di **Frame 18**. Ini bertujuan agar player baru menerima damage Ketika masih berada di area serang zombie pada Frame ke 18.

- **Pindahkan Animator Controller.**

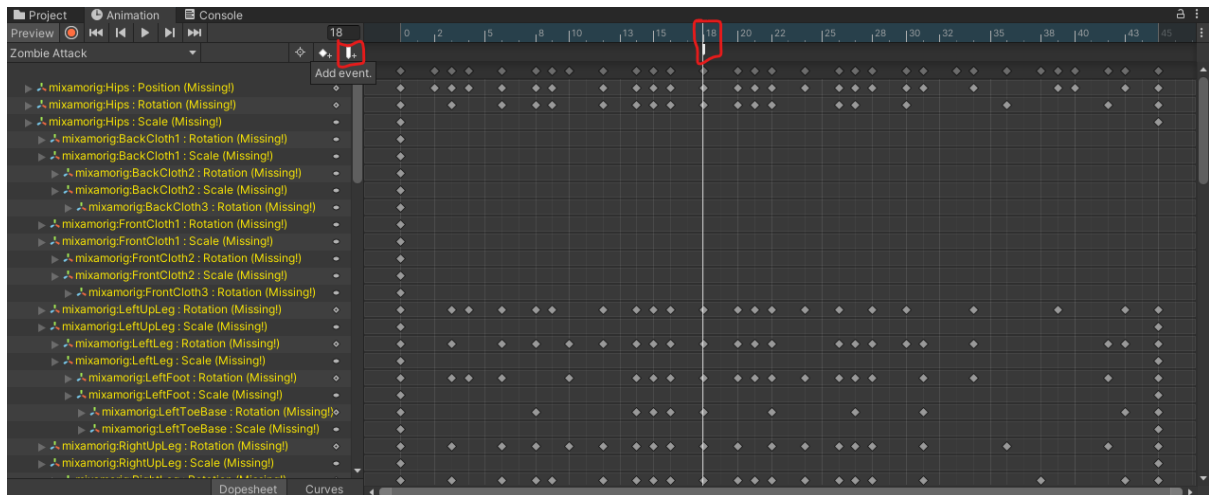
Pindahkan component Animator yang berada di gameobject **“Skeletonzombie T Avelange”**, ke GameObeject **“Enemy”**. Hal ini bertujuan karena **Event** membutuhkan GameObject yang menampung Script. Cara mudahnya :

RightClick Component – Copy Component – Seleksi GameObject tujuan – Pada inspector LeftClick – Paste Component as New – Hapus Component asal

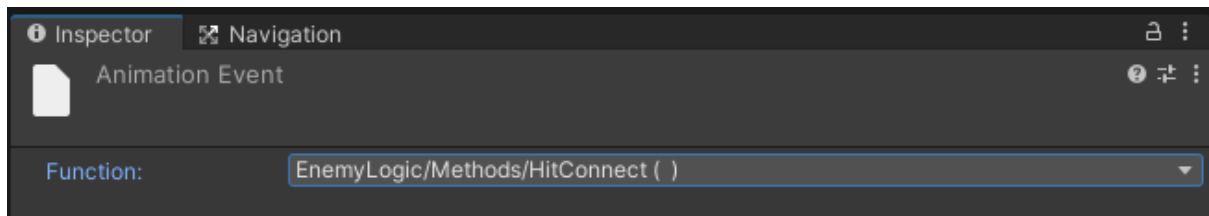


- **Add Event HitConnect**

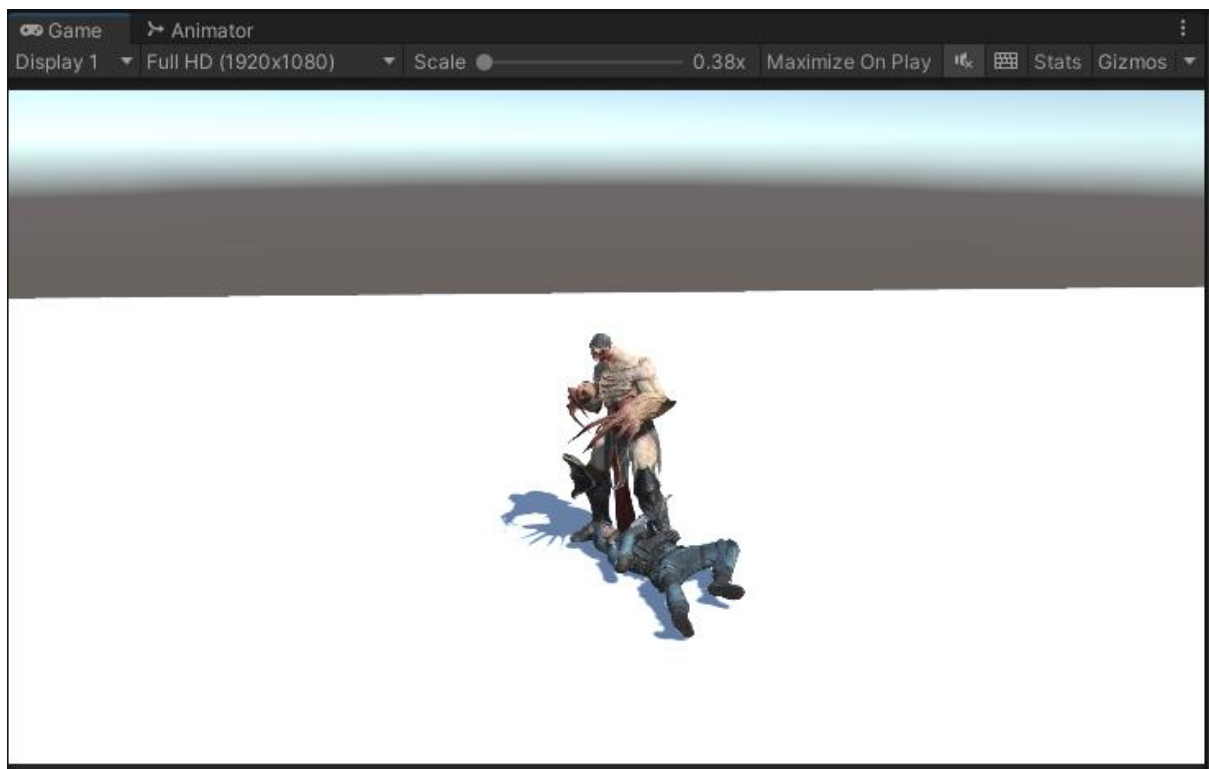
Selanjutnya adalah menambahkan Event di frame ke 18 Zombie attack dengan Event memanggil Method Hit Connect. Caranya adalah buka tab Animation, lalu click icon Add event Ketika berada di Frame 18. Jika sudah akan seperti gambar di bawah ini.



- **Tambahkan Function HitConnect diinspector.**



Lakukan Pengetesan Player diserang Enemy, jika pada frame ke 18 player menerima damage dan player mati Ketika health = 0, maka konfigurasi dan kodingan benar, jika belum lakukan pengecekan ulang!



2. Back to Initial Position.

Logika selanjutnya adalah enemy kembali ke tempat semula ketika **Player** keluar dari **Chase Range**. Tulis beberapa baris Script berikut.

- **Deklarasi Dan Inisialisasi Variable Baru**

```
private float DistancetoTarget;
private float DistancetoDefault;
private Animator anim;
Vector3 DefaultPosition;

Unity Message | 0 references
private void Start()
{
    agent = this.GetComponent<NavMeshAgent>();
    anim = this.GetComponentInChildren<Animator>();
    anim.SetFloat("Hitpoint", hitPoints);
    DefaultPosition = this.transform.position;
}
```

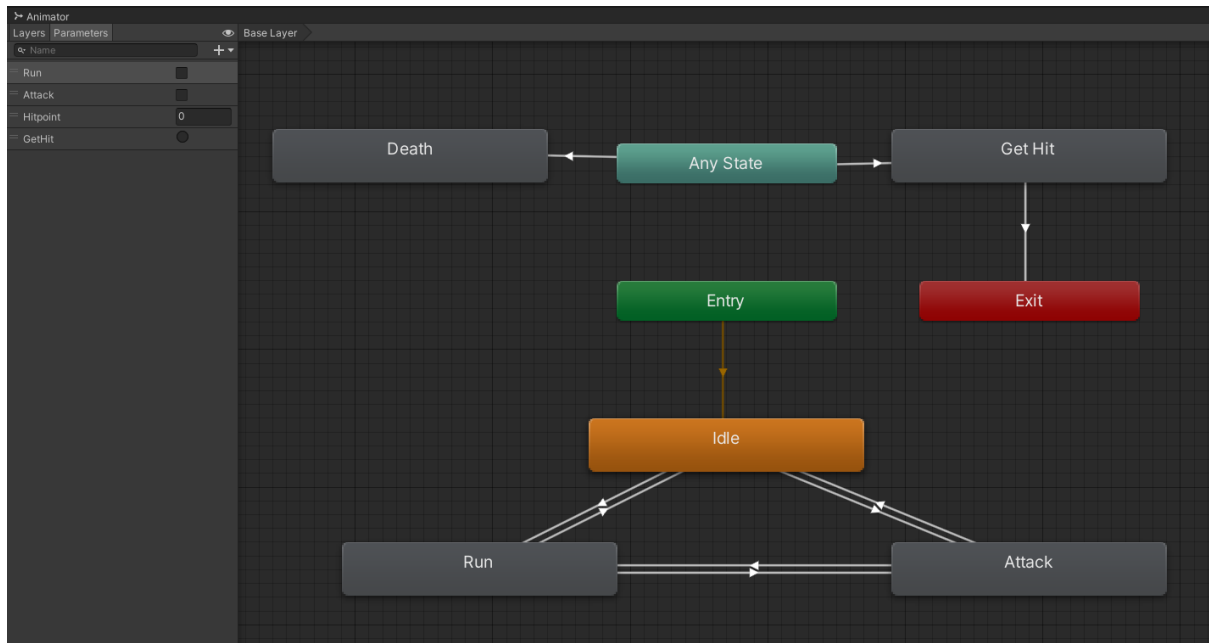
- Tambahkan beberapa baris kode pada **method Update**.

```
Unity Message | 0 references
private void Update()
{
    DistancetoTarget = Vector3.Distance(target.position, transform.position);
    DistancetoDefault = Vector3.Distance(DefaultPosition, transform.position);

    if (DistancetoTarget <= ChaseRange && hitPoints != 0)
    {
        FaceTarget(target.position);
        if (DistancetoTarget > agent.stoppingDistance + 2f)
        {
            ChaseTarget();
        }
        else if (DistancetoTarget <= agent.stoppingDistance)
        {
            Attack();
        }
    }
    else if (DistancetoTarget >= ChaseRange * 2)
    {
        agent.SetDestination(DefaultPosition);
        FaceTarget(DefaultPosition);
        if (DistancetoDefault <= agent.stoppingDistance)
        {
            Debug.Log("Time to stop");
            anim.SetBool("Run", false);
            anim.SetBool("Attack", false);
        }
    }
}
```

Jika sudah, Save Script dan Kembali ke unity, lakukan pengetesan apakah enemy Kembali ke posisi awal ketika player menjauh saat dikejar enemy. jika iya maka konfigurasi dan script sudah benar, jika masih belum lakukan pengecekan ulang!

3. Melengkapi Animasi Enemy



Tambahkan 2 state baru pada animator Enemy seperti gambar di atas, dan berikut adalah baris kode pemicu state **Death** dan **GetHit**. Pastikan Ketika kondisi **HitPoint = 0** state yang terpanggil adalah **Death**, bukan **GetHit**. Dan tambahkan delay destroy GameObject selama 3 detik untuk menunggu animasi Death selesai terlebih dahulu.

```
1 reference
public void TakeDamage(float damage)
{
    hitPoints -= damage;
    anim.SetTrigger("GetHit");
    anim.SetFloat("Hitpoint", hitPoints);
    if (hitPoints <= 0)
    {
        Destroy(gameObject, 3f);
    }
}
```

4. Selesai Untuk Tugas Modul 6

Tugas Modul 6 Dikatakan Selesai jika Praktikan menyelesaikan **Add Animation Event, Back to initial Position**, dan menambahkan animasi Enemy **Death & GetHit**.